

SmartClinic Microservices

SOA and Microservices - Mini Project

Application: clinic management with patients, doctors, appointments and notifications

Team repository: smartclinic-microservices

Work done by :

Amira Farrah

Raed Alouani

1. Project idea

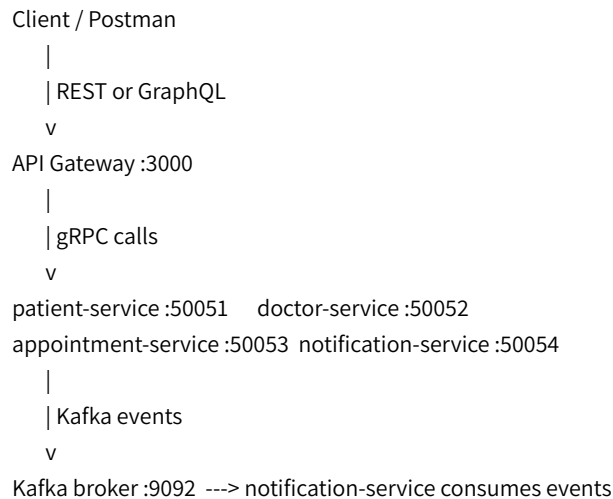
SmartClinic is a simple clinic management project. We divided it into small services, and each service has its own work to do. The user does not use the services directly. All requests go first to the API Gateway. After that, the Gateway sends the request to the right service using gRPC.

In the project, we can create patients, create doctors, create appointments, and update appointment status. When something important happens, like creating a patient or cancelling an appointment, the system saves a notification automatically.

Part	Role
Patient service	Keeps patient information and sends a Kafka event when a patient is created.
Doctor service	Keeps doctor information and gives simple availability/search operations.
Appointment service	Creates and updates appointments. It sends Kafka events for creation and cancellation.
Notification service	Consumes Kafka events and saves notifications.
API Gateway	Entry point for REST and GraphQL. It calls all services with gRPC.

2. Architecture

The architecture follows the required microservice style: API Gateway, independent microservices, gRPC communication, Kafka broker, and one database per service.



Component	Port	Description
API Gateway	3000	REST + GraphQL interface for the client
Patient service	50051	gRPC service for patient CRUD and search
Doctor service	50052	gRPC service for doctor CRUD, specialty search and availability
Appointment service	50053	gRPC service for appointment CRUD and status update
Notification service	50054	gRPC service for notifications + Kafka consumer
Kafka broker	9092	Asynchronous communication between services

3. How the project was created

We built the project step by step. The idea was to keep the code simple and easy to test instead of putting everything in one server.

1. We created the main folders: api-gateway, patient-service, doctor-service, appointment-service, notification-service and proto.
2. We wrote a .proto file for each service. These files are the contract between the API Gateway and the microservices.
3. For each service, we created a small SQLite database and a db.js file to connect to it.
4. We implemented the gRPC server of each microservice in src/server.js.
5. In the API Gateway, we created gRPC clients to call the services.
6. We added REST routes in the API Gateway, for normal CRUD operations.
7. We added GraphQL typeDefs and resolvers, for flexible queries.
8. We added Docker Compose for Kafka.
9. We added Kafka producers in patient-service and appointment-service.
10. We added a Kafka consumer in notification-service.
11. Finally, we tested the full scenario with Postman.

4. gRPC and Protocol Buffers

gRPC is the main internal communication method in the project. The API Gateway calls the services through gRPC. The contracts are written in the proto folder.

Proto file	Service
proto/patient.proto	PatientService
proto/doctor.proto	DoctorService
proto/appointment.proto	AppointmentService
proto/notification.proto	NotificationService

4.1 Main gRPC methods

Service	Methods
PatientService	CreatePatient, GetPatient, ListPatients, UpdatePatient, DeletePatient, SearchPatients
DoctorService	CreateDoctor, GetDoctor, ListDoctors, UpdateDoctor, DeleteDoctor, SearchDoctorsBySpecialty, CheckDoctorAvailability
AppointmentService	CreateAppointment, GetAppointment, ListAppointments, UpdateAppointmentStatus, DeleteAppointment, ListAppointmentsByPatient, ListAppointmentsByDoctor
NotificationService	CreateNotification, GetNotification, ListNotifications, ListNotificationsByPatient, MarkNotificationAsRead

The separation is clear: the proto files define the interface, and the server.js files contain the implementation. This makes it easier to change the client side without changing the contract randomly.

5. REST API

REST is used for normal operations like create, list, update, delete and search. The base URL is <http://localhost:3000>.

5.1 Patient endpoints

Method	Endpoint	Use
POST	/patients	Create a patient
GET	/patients	List patients
GET	/patients/search?name=Test	Search patients by name
GET	/patients/:id	Get patient by id
PUT	/patients/:id	Update patient
DELETE	/patients/:id	Delete patient

5.2 Doctor endpoints

Method	Endpoint	Use
POST	/doctors	Create a doctor
GET	/doctors	List doctors
GET	/doctors/search?specialty=Cardiology	Search doctors by specialty
GET	/doctors/:id	Get doctor by id
PUT	/doctors/:id	Update doctor
DELETE	/doctors/:id	Delete doctor
GET	/doctors/:id/availability?date=2026-05-20	Check doctor availability

5.3 Appointment endpoints

Method	Endpoint	Use
POST	/appointments	Create appointment
GET	/appointments	List appointments
GET	/appointments/:id	Get appointment by id
GET	/appointments/patient/:patientId	List appointments by patient
GET	/appointments/doctor/:doctorId	List appointments by doctor
PUT	/appointments/:id/status	Update appointment status
DELETE	/appointments/:id	Delete appointment

5.4 Notification endpoints

Method	Endpoint	Use
POST	/notifications	Create notification manually
GET	/notifications	List notifications
GET	/notifications/:id	Get notification by id
GET	/notifications/patient/:patientId	List notifications by patient
PUT	/notifications/:id/read	Mark notification as read

6. GraphQL

GraphQL is exposed from the API Gateway at <http://localhost:3000/graphql>. It is useful when the client wants to select only some fields, for example only id, name and email.

Type	Main fields
Patient	id, full_name, email, phone, birth_date, created_at
Doctor	id, full_name, specialty, email, phone, created_at
Appointment	id, patient_id, doctor_id, date, reason, status, created_at
Notification	id, patient_id, type, message, is_read, created_at

6.1 Example GraphQL query

```

query {
  patients {
    id
    full_name
    email
  }
}
```

6.2 Example GraphQL mutation

```

mutation {
  createPatient(
    full_name: "GraphQL Patient"
    email: "graphql.patient@gmail.com"
    phone: "99999999"
    birth_date: "2001-01-01"
  ) {
    id
    full_name
    email
  }
}
```

7. Kafka events

Kafka is used for asynchronous communication. We did not use Kafka for every request. We used it only when another service needs to react to an event without blocking the original request.

Topic	Producer	Consumer	Trigger
patient.created	patient-service	notification-service	A new patient is created
appointment.created	appointment-service	notification-service	A new appointment is created
appointment.cancelled	appointment-service	notification-service	Appointment status is changed to CANCELLED

7.1 Kafka messages

Topic	Main data sent
patient.created	patient_id, full_name, email, phone, birth_date, created_at
appointment.created	appointment_id, patient_id, doctor_id, date, reason, status
appointment.cancelled	appointment_id, patient_id, doctor_id, date, status

7.2 Tested Kafka scenario

12. Create patient from Postman. Patient service saves it and sends patient.created.
13. Notification service consumes the event and saves a welcome notification.
14. Create appointment. Appointment service sends appointment.created.
15. Notification service saves an appointment notification.
16. Cancel appointment by calling PUT /appointments/:id/status with status CANCELLED.
17. Appointment service sends appointment.cancelled and notification service saves the cancellation notification.

8. Databases

Every service has its own SQLite database file. The databases are separated, so the services do not share tables directly.

Service	Database	Main table
patient-service	patients.sqlite	patients
doctor-service	doctors.sqlite	doctors
appointment-service	appointments.sqlite	appointments
notification-service	notifications.sqlite	notifications

The SQLite files are ignored in Git because they are generated locally when running the services.

9. Installation and run instructions

Before running the project, Docker Desktop must be open and running because Kafka is started with Docker Compose.

9.1 Start Kafka

```
docker compose up -d
docker ps
```

9.2 Install dependencies

```
cd patient-service
npm install
cd ..
```

```
cd doctor-service
npm install
cd ..
```

```
cd appointment-service
npm install
cd ..
```

```
cd notification-service
npm install
cd ..
```

```
cd api-gateway
npm install
cd ..
```

9.3 Start services

```
# terminal 1
cd patient-service
npm start

# terminal 2
cd doctor-service
npm start

# terminal 3
cd appointment-service
npm start

# terminal 4
cd notification-service
npm start

# terminal 5
cd api-gateway
npm start
```

10. Test examples

10.1 Create patient

POST http://localhost:3000/patients

```
{
  "full_name": "Test Patient",
  "email": "test.patient@gmail.com",
  "phone": "12345678",
  "birth_date": "2000-01-01"
}
```

10.2 Create appointment

POST http://localhost:3000/appointments

```
{
  "patient_id": "PASTE_PATIENT_ID",
  "doctor_id": "PASTE_DOCTOR_ID",
  "date": "2026-05-20",
  "reason": "General check"
}
```

10.3 Cancel appointment

PUT `http://localhost:3000/appointments/APPOINTMENT_ID/status`

```
{ "status": "CANCELLED" }
```

10.4 Expected logs

Patient Kafka producer connected.

Kafka event sent to topic: patient.created

Appointment Kafka producer connected.

Kafka event sent to topic: appointment.created

Kafka event sent to topic: appointment.cancelled

Notification Kafka consumer started.

Notification saved: Welcome Test Patient, your patient profile was created.

Notification saved: Your appointment was created for 2026-05-20.

Notification saved: Your appointment <id> was cancelled.

11. Project organization

Folder	Content
api-gateway	REST routes, GraphQL schema/resolvers, gRPC clients
patient-service	Patient gRPC server, SQLite db, Kafka producer
doctor-service	Doctor gRPC server and SQLite db
appointment-service	Appointment gRPC server, SQLite db, Kafka producer
notification-service	Notification gRPC server, SQLite db, Kafka consumer
proto	All proto contracts
postman	Requests used to test the API
docker-compose.yml	Kafka broker setup

12. Requirement check

Requirement	Where it is done
At least three microservices	patient, doctor, appointment, notification services
Node.js only	All services and API Gateway are Node.js
API Gateway	api-gateway on port 3000
REST endpoints	api-gateway/src/routes
GraphQL	api-gateway/src/graphql
gRPC communication	proto files + gRPC clients/servers
Kafka broker	docker-compose.yml
Kafka producers and consumers	patient-service, appointment-service, notification-service
Separate databases	SQLite database per service
Test interface	Postman collection

The project is now ready for the next part: improving final README text, adding screenshots if needed, and preparing the presentation.